

Verification Method of UML Diagrams Using Large Language Models

Masaaki Negishi

Graduate School of Engineering and Science,
Shibaura Institute of Technology, Saitama, Japan
bp21063@shibaura-it.ac.jp

Kenji Hisazumi

Shibaura Institute of Technology, Saitama, Japan
hisazumi@shibaura-it.ac.jp

Abstract—UML state diagrams are increasingly used in secondary education, such as the Japanese Informatics I course, yet students often struggle with dynamic behavior even after mastering surface notation. Existing execution environments assume carefully prepared digital models in dedicated tools, which can be demanding for novices. This work-in-progress proposes an LLM-based approach that generates executable code directly from images of hand-drawn UML state diagrams, allowing students to experiment with runtime behavior while keeping the familiar practice of sketching by hand. We implement prototypes for smartphone and web environments and evaluate conversion accuracy on diagrams of varying complexity. Preliminary experiments show useful accuracy on low-complexity diagrams and confirm that the system does not silently fix intentionally faulty diagrams, suggesting that this approach is promising for supporting novices in learning dynamic behavior.

Index Terms—UML, Large Language Models, Hand-drawn Diagrams, Secondary Education, Dynamic Behavior Verification, Code Generation

I. INTRODUCTION

UML state diagrams are increasingly used in software engineering education and have recently entered secondary-school curricula such as the Japanese “Informatics I” course. Students are expected not only to read and draw these diagrams but also to reason about the dynamic behavior of systems, for example which states are reachable and how guards and actions interact.

In practice, however, novices often master surface notation while still struggling with runtime behavior. Classroom activities tend to focus on syntax checks or manually constructed trace tables, and existing execution environments assume carefully prepared digital models in dedicated tools. For many secondary-school learners, moving from hand-drawn diagrams on paper or slides to such tools introduces additional overhead and can become a barrier rather than a support for learning.

Recent progress in large language models (LLMs) suggests a different approach. Instead of asking learners to recreate their diagrams in a modeling editor, an LLM could interpret hand-drawn UML state diagrams directly from images and generate executable code that reflects the depicted behavior. This would allow students to experiment with runtime behavior while keeping the familiar practice of sketching diagrams by hand.

This work-in-progress investigates the feasibility of such an approach as a basis for novice support. We implement an LLM-based pipeline that generates executable code or error reports from images of UML state diagrams under the

constraint that the LLM may only use information visible in the drawing. We realize this pipeline in two prototypes: a smartphone app that provides code and, when code generation fails, concise revision guidance, and a web-based prototype that integrates code generation and execution and adopts compiler-like feedback that reports only fatal diagram errors. We evaluate conversion accuracy and failure patterns as an initial step toward assessing whether this approach is reliable enough for classroom use and discuss directions for future classroom trials.

- We propose an LLM-based pipeline that takes a hand-drawn UML state diagram as an image, uses the LLM to reconstruct its structure, and generates executable Java code or an explicit error report, while restricting the LLM to use only the information visible in the diagram.
- We implement a smartphone-based prototype that provides both code generation and, when code generation fails, natural-language revision guidance, and we evaluate its conversion accuracy, suppression of erroneous “auto-completion”, and the quality of its guidance.
- We develop a web-based prototype (UMLReviewWebApp) that integrates code generation and execution in the browser and intentionally shifts to a more compiler-like behavior, generating code that faithfully reflects the input diagram and reporting only fatal diagram errors.

We report preliminary experimental results from the smartphone prototype and outline a lightweight but convincing evaluation design for the web-based prototype that can be completed in roughly one hour. Our goal is to clarify which ranges of diagram complexity are technically feasible for classroom use and to discuss design trade-offs between assistant-style and compiler-like feedback.

II. BACKGROUND AND MOTIVATION

Model-driven and executable modeling tools for education aim to make system behavior tangible by allowing learners to simulate or run their models. Prior work has used domain-specific notations and tools to convert diagrams into executable artifacts, for example by transforming hand-drawn control-loop sketches on whiteboards into simulation models for control education [1] or by using model-driven development tools such as Clooca in object-oriented modeling courses [2]. These approaches have demonstrated that running models can deepen

understanding, but they typically assume a dedicated modeling environment and support only a limited set of diagram types and domains.

In UML-based education, tools commonly focus on creating and checking diagrams within a specific platform. While such platforms provide precise syntax and automated checks, they are often designed with professional users in mind and expose complex user interfaces, multiple configuration steps, and a large number of features that novices do not immediately need. For secondary-school learners who are just beginning to encounter UML, the combination of tool complexity and setup effort can make existing environments hard to adopt, especially when the goal is simply to experiment with a few small examples.

Another design dimension concerns how feedback is presented. Assistant-style tools attempt to infer the learner's intended model and provide hints or revision suggestions when inconsistencies are detected. This can be helpful but may also produce verbose or speculative feedback that overwhelms novices or encourages them to follow suggestions without reflecting on semantics. In contrast, compiler-like tools adopt a stricter stance: they either produce executable artifacts that faithfully reflect the given input or reject it with concise error messages. In this work we explore LLM-based support for hand-drawn UML diagrams under both perspectives, using an assistant-style smartphone prototype and a compiler-like web prototype, and investigate how far these designs can go in supporting novice learners.

III. LLM-BASED VERIFICATION PIPELINE AND PROTOTYPES

A. Overall pipeline

Figure 1 gives an overview of our LLM-based pipeline. A learner first captures a hand-drawn UML state diagram as an image, either by taking a photo of a paper or whiteboard drawing on a smartphone or by uploading a sketch from a tablet or drawing tool. The image is normalized in resolution and contrast and then sent to an LLM with a prompt that instructs it to reconstruct the states, transitions, events, guards, and actions that are visible in the diagram.

For the experiments reported in this paper, we used Google's Gemini 2.5 Flash multimodal model via its public API.

The LLM is constrained to use only information present in the image and is asked either to emit executable code that simulates the diagram's behavior or, if it detects fatal well-formedness problems, to return a structured error report instead of code. When code is produced, a small driver program is included so that learners can supply event sequences and observe the resulting state transitions. Execution is performed in an external environment on the smartphone or directly in the browser in the web prototype.

B. Smartphone prototype with assistant-style feedback

The smartphone prototype targets Android devices and acts as an assistant-style tool. Users take a photo of a hand-drawn state diagram, select the diagram type, and submit the

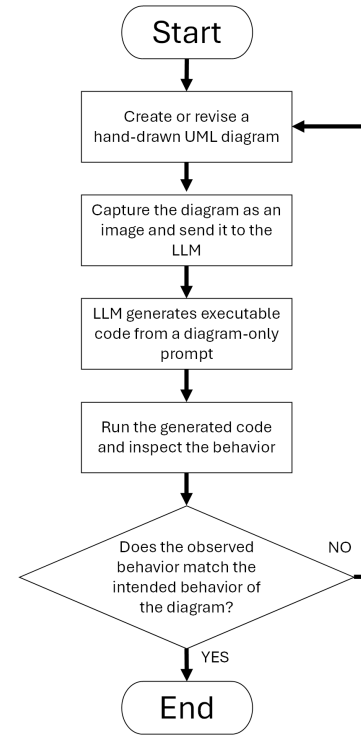


Fig. 1. Overview of the LLM-based pipeline for hand-drawn UML state diagrams.

image to the backend. If the LLM judges the diagram to be interpretable as a UML state machine, it generates executable code together with a short textual explanation of the behavior. If it detects fatal issues that make code generation impossible, it does not attempt to repair the diagram; instead, it returns concise revision guidance in natural language, such as adding an initial state or connecting isolated states. Learners can iteratively revise their diagrams and rerun the pipeline until code generation succeeds.

C. Web prototype with compiler-like feedback

To better match classroom settings in which learners use laptops or school PCs, we implemented a web-based prototype that applies the same pipeline on the server side but offers a different feedback style. The web app lets users upload an image of a hand-drawn state diagram and then displays the generated code and execution trace directly in the browser.

In this prototype, the LLM is instructed to behave in a more compiler-like manner. As long as the diagram is syntactically well-formed as a UML state machine, the system tries to generate code that faithfully reflects the given diagram, even if the result appears semantically odd. Only when the diagram violates basic well-formedness constraints does the system reject the input and return an error message instead of code. This design aims to reduce noisy or speculative hints, emphasize the learner's responsibility for producing well-formed diagrams, and still enable rapid experimentation with dynamic behavior in a browser-based environment.

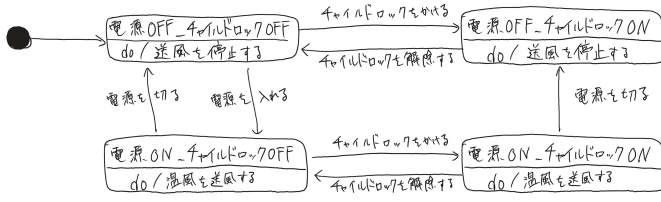


Fig. 2. Sample hand-drawn UML state diagram of a heater with a child-lock feature.

TABLE I
CONVERSION OUTCOMES FOR VALID DIAGRAMS GROUPED BY CC.

Subset	# diag.	Smartphone (corr.)	Web (corr.)
All	10	21/37	26/40
$CC \leq 6$	7	18/26	20/28
$CC > 6$	3	3/11	4/12

IV. EVALUATION

We evaluated the smartphone and web prototypes on a small set of hand-drawn UML state diagrams to obtain initial evidence about conversion accuracy and failure behavior.

A. Setup

We prepared ten valid hand-drawn UML state diagrams covering a range of cyclomatic complexities (CC) typical for introductory examples. Figure 2 shows a sample hand-drawn UML state diagram of a heater with a child-lock feature. The model abstracts the behavior so that, for example, the power can always be turned off, but cannot be turned on while the child lock is enabled. In addition to these valid diagrams, we created erroneous variants from four of them by deleting states and/or transitions so that parts of the diagram became unreachable. Each image (valid or erroneous) was submitted up to four times to each prototype to account for the stochastic behavior of the LLM.

For valid diagrams we executed a small set of test event sequences on the generated code and classified each run as *correct* if the observed behavior matched the intended behavior of the base diagram, or *incorrect* otherwise. For diagrams with intentionally inserted errors we did not use the labels *correct* and *incorrect*; instead we checked whether a prototype tried to compensate for the injected errors. We say that a prototype *rejected* an erroneous diagram if, across all runs for that diagram, it never generated code that compensates for the injected errors. In practice, this means that the smartphone prototype only reported errors without producing code, and the web prototype, when it produced code, preserved the faulty structure rather than repairing it.

B. Valid diagrams

Table I summarizes the conversion outcomes for the ten valid diagrams. Both prototypes are more reliable on diagrams with lower CC than on more complex ones. For lower-CC diagrams, the smartphone prototype achieved a practically useful success rate, and the web prototype showed comparable behavior while applying the stricter, compiler-like feedback

TABLE II
OUTCOMES FOR DIAGRAMS WITH INTENTIONALLY INSERTED ERRORS. FOR EACH DIAGRAM WE REPORT THE CC OF THE CORRESPONDING VALID BASE DIAGRAM.

Diagram	Base CC	Smartphone	Web
E1	5	rejected	rejected
E2	5	rejected	rejected
E3	6	rejected	rejected
E4	6	rejected	rejected

strategy. For higher-CC diagrams, both prototypes produced incorrect or rejected outputs more frequently, suggesting that the current pipeline is best suited for relatively simple examples.

C. Erroneous diagrams

Table II shows the outcomes for the four diagrams with intentionally inserted errors. All four diagrams were rejected by both prototypes in the sense defined in the setup: we did not observe any cases in which either prototype generated code that compensates for the injected errors and thus silently “fixed” the faulty inputs. Under our prompting strategy the LLM therefore behaves conservatively on clearly faulty inputs and does not mask structural problems in the hand-drawn diagrams, which is important if the tool is to be used for formative assessment in classroom settings.

V. DISCUSSION AND FUTURE WORK

Our preliminary findings indicate that LLM-based code generation from hand-drawn UML state diagrams is technically feasible for diagrams with limited complexity and that, under our prompting strategy, the LLM can avoid silently “fixing” diagrams that contain intentional errors. At the same time, the evaluation suggests that diagrams with orthogonal or composite states and transitions with long, heavily annotated labels remain challenging, and that simple metrics such as CC do not fully capture this difficulty.

The shift from an assistant-style smartphone prototype to the compiler-like feedback of the web prototype reflects a deliberate design choice: we prioritize faithfulness to the given diagram and aim to reduce speculative hints that might discourage reflection. As future work, we plan to extend the web-based prototype to a wider range of hand-drawn examples and diagram types, and to conduct classroom studies with teachers and students to investigate learning outcomes, usability, and how compiler-like rejection can be combined with concise, age-appropriate hints that support learning without inviting overreliance on the tool.

REFERENCES

- [1] D. Erdem, A. Beke, and T. Kumbasar, “A deep learning-based pipeline for teaching control theory: Transforming feedback control systems on whiteboard into MATLAB,” *IEEE Access*, vol. 8, pp. 84631–84641, 2020.
- [2] S. Akayama *et al.*, “Using model-driven development tools for object-oriented modeling education,” in *Proc. EduSymp@MoDELS*, 2013.